

Maven Java Application Development

You are given the following GitHub repository link:

<https://github.com/Kumbhambhargavi75/XXXXXX/>

1. Import the project into Maven environment

Steps to import :

Open Eclipse

1. Launch **Eclipse IDE** (preferably Eclipse IDE for Enterprise Java Developers).

2. Clone Repository

Go to the menu:

File → **Import** → **Git** → **Projects from Git (with smart import)** → Next.

Choose **Clone URI** → Next.

Enter the **GitHub repository URL** (SSH or HTTPS):

- Example (HTTPS):
- <https://github.com/username/repository.git>

Select the **branches** you want (usually **main** or master) → Next.

Choose the **directory** where Eclipse will store the repo → Finish.

Now you can see the imported project in the eclipse under project explorer

2. Questions on pom.xml (check the pom.xml correct or not and Resolve dependencies using pom.xml)
3. Build the project to generate the **WAR/JAR file**
4. Verify the generated artifact in the `target/` folder
5. Solve the SBQ's (.M2repository, junit,test,build,finalname Etc related to pom.xml))

1. Dockerfile Creation

Create a Dockerfile in IDE Eclipse

Go to your project → src → main → webapp → rightclick → goto new → select file → enter the filename as Dockerfile

```
FROM tomcat:9.0
COPY target/*.war /usr/local/tomcat/webapps/ROOT.war
EXPOSE 7089
CMD ["catalina.sh", "run"]
```

Explanation

- 1.FROM tomcat:9.0
Uses the official Tomcat 9 base image.
- 2.COPY target/*.war /usr/local/tomcat/webapps/ROOT.war
Copies your WAR file(s) (from Maven/Gradle build) into Tomcat's webapps folder.
When Tomcat starts, it will automatically unpack and deploy them.
By naming it ROOT.war, Tomcat deploys it as the **root web application** (/), so you can access it at: http://localhost:<mapped-port>/
- 3.EXPOSE 8080

.Important: This does **not** actually change Tomcat's port.
.Tomcat by default listens on **8080**, unless you edit server.xml.

1. CMD ["catalina.sh", "run"]
.CMD defines the **default command** to run when the container starts.
.catalina.sh is Tomcat's startup script.
.The argument "run" tells it to:
Start Tomcat **in the foreground** (inside the container).

For java app

```
FROM redis:latest
CMD ["redis-server"]
```

Git and GitHub

Task: Work with Git and GitHub as follows:

Open gitbash in ECLIPSE ----Rightclick on your mavenproject-->show in local terminal -->git bash

2. Initialize a Git repository and add your project files
3. Set global config,username and user email
4. Solve the given **SQB (Short Question Based)** Git tasks (e.g., branch, merge, revert,stash,restore,merge conflict,gitignore,clone) Push your Maven project to GitHub

Gitbash open in the eclipse with the path of your project.

```
$git init
```

```
$git add .
```

```
$git commit -m "first commit"
```

```
$git config --global user.name "your user name"
```

```
$git config --global user.email "your email id"
```

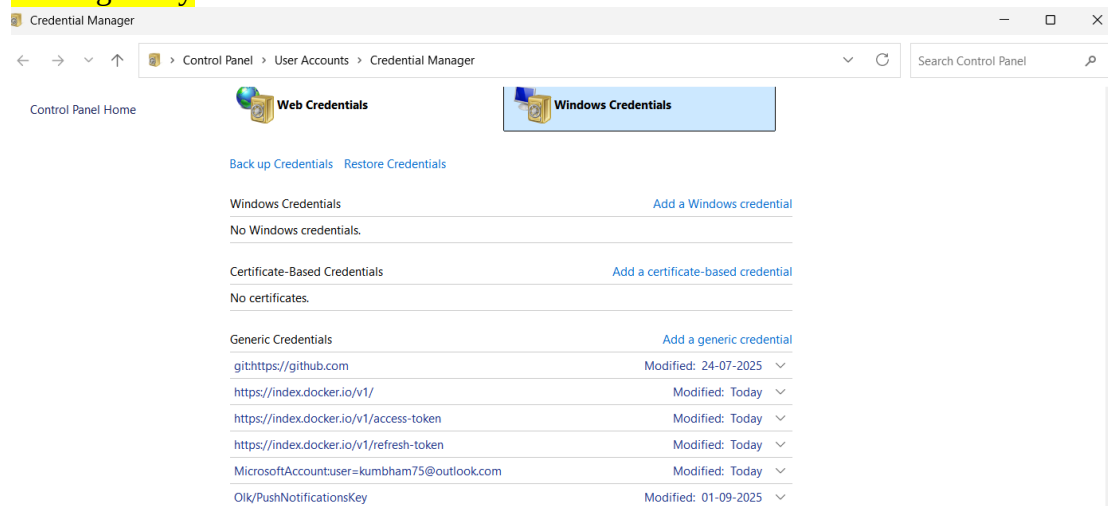
Solve the given **SQB's**

Push your Maven project to your GitHub

Go to your github crate a repository for mavaneproject name

Come to git bash and push

Before pushing Check the windows credentials and remove the git hub accounts/login existing if any



```
$git branch -M main
```

```
$git remote add origin1 https://github.com/Kumbhambhargavi75/LMSWEBP.git
```

```
$git push -u origin1 main
```

Note : if you are trying to push for the given github, access denied getting so change origin(trying to push to the owner) to origin1 or other-name and clear the github accounts in windows credentials if exist

OR push by setting URL also

```
git remote set-url origin https://github.com/YOURGITHUBUSERNAME/LMSWEBP.git
```

Now Refresh your github and check files exist or not

Docker

Note: Containerize your Maven project using Docker.(open the docker desktop and docker hub account)

EXAMPLE Tasks:

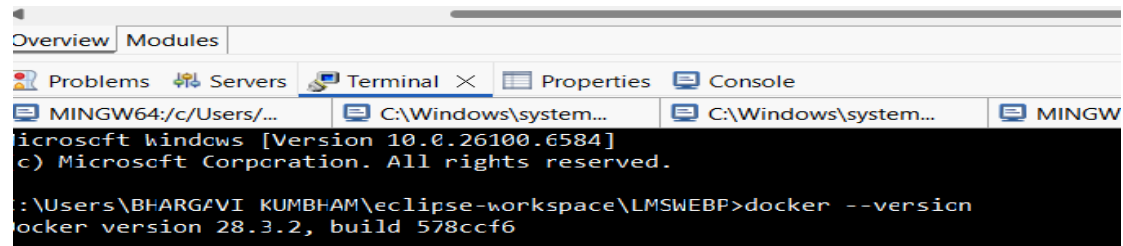
1. Creation of Dockerfile and build a docker image and push it to dockerhub
2. Run the image in a container and expose to the particular port number(ex:9098)

Open gitbash in ECLIPSE ----Rightclick on your mavenproject-->show in local terminal --

>terminal

OR can do in windows power shell OR gitbash

Here I opened in eclipse only ----Check the docker version exist or not

A screenshot of the Eclipse IDE's integrated terminal. The terminal window has tabs for 'Overview', 'Modules', 'Problems', 'Servers', 'Terminal', 'Properties', and 'Console'. The 'Terminal' tab is active, showing a command prompt with the command 'docker --version' and its output 'Docker version 28.3.2, build 578ccf6'. The terminal's title bar shows the path 'C:\Windows\system...'.

```
Microsoft Windows [Version 10.0.26100.6584]
(c) Microsoft Corporation. All rights reserved.

:\Users\BHARGAVI KUMBHAM\eclipse-workspace\LMSWEBP>docker --version
Docker version 28.3.2, build 578ccf6
```

>docker ps -a

Displays the list of images running currently or previously or stopped

>docker image ls

displays the list of docker images in the local server

>docker login

login to docker hub with required credentials

SBQ's (pull the image Locally/Hub and run a container,check the list of containers, stop the container, start the container, run it in interactive mode,build the image ,run it on localhost by given port number)

2. Image Building

- o Build the Docker image using `docker build -t <image_name> .`

Example : `docker build -t lmsimage .`

- a. `docker build` → tells Docker to build an image.
- b. `-t <image_name>` → tags the image with a name (so you can run it later).
- c. Example: `-t lmsimage`
- d. Or if pushing to Docker Hub: `-t kumbhambhargavi/lmsimage:latest`
- e. `.` → build context = the **current directory** (must contain your Dockerfile).

```

d1372475a1b1 mysql:5.7 "docker-entrypoint.s..." 3 weeks ago Up 3 days 3306/tcp, 33060/tcp
ydockerproject-db-1

C:\Users\BHARGAVI KUMBHAM\eclipse-workspace\MSWEBP>docker build -t lmsimage .
[*] Building 2.6s (7/7) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 148B
=> [internal] load metadata for docker.io/library/tomcat:9.0
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 2.62kB
=> CACHED [1/2] FROM docker.io/library/tomcat:9.0@sha256:025449e15189de00d632417abc0573f927aab1728205b4e368ddff92f4f9b48f

```

RUN THE IMAGE

```
$docker run -d -p 7089:8080 --name lmcontainer lmsimage
```

- d → run in detached mode (in background).
- p 7089:8080 → map **host port 7089** → tomcat **container port 8080**.
- name lmcontainer → give the container a friendly name (lmcontainer).
- lmsimage → start from your image.

Now check <http://localhost:7089/>
(opens the MSWEBP APP)

3. Push to Docker Hub

- Tag and push the created image to your Docker Hub account.

Docker Hub images must be tagged as:

```
<dockerhub-username>/<repository-name>:<tag>
```

Ex: `docker tag lmsimage kumbhambhargavi/lmsimage:latest`

```
docker push kumbhambhargavi/lmsimage:latest
```

This will upload your image to your Docker Hub repo.

```

C:\Users\BHARGAVI KUMBHAM\eclipse-workspace\MSWEBP>docker run -d -p 7089:8080 --name lmcontainer lmsimage
5dba34bbcab9ab1d84e92f180a88a717b9768449303dcd27ff75f1986ba2305b

C:\Users\BHARGAVI KUMBHAM\eclipse-workspace\MSWEBP> docker tag lmsimage kumbhambhargavi75/lmsimage:latest

C:\Users\BHARGAVI KUMBHAM\eclipse-workspace\MSWEBP>docker push kumbhambhargavi75/lmsimage:latest
The push refers to repository [docker.io/kumbhambhargavi75/lmsimage]
cee05d3a8171: waiting
82d8ab29e0aa: waiting
4f4fb700ef54: waiting

```

Go to <https://hub.docker.com/repositories> → you can see lmsimage.

PULL IMAGE OF PYTHON AN RUN IT IN INRERACTIVE MODE

A.How to run Ubuntu Image

```
docker run [image name]
```

```
>docker run Ubuntu
```

executing a docker image in the local server

```
docker run -it <container id / name> bash
```

running a ubuntu container in interactive mode.... Using bash commands to ammend the existing image

```
C:\Users\BHARGAVI KUMBHAM>docker run -it ubuntu bash
root@779228eb144b:/# echo Dr.K.Bhargavi
Dr.K.Bhargavi
root@779228eb144b:/# pwd
/
root@779228eb144b:/# ls
bin boot dev etc home lib lib64 media mnt opt proc root run sbin srv sys tmp usr var
root@779228eb144b:/# exit
exit
```

B.Pushing the images from local repository to docker hub

Tag and push the image:

docker push <image name> push the image to the docker hub repository

C.stops running container

docker stop containerid

>docker stop 96724fc65bd8

D. remove container

>docker rm 96724fc65bd8

Answer short questions related to Docker commands, image vs. container, etc

```
mvn clean package
mvn test
mvn clean install
mvn clean package -DskipTests
mvn dependency:tree
```

```
git status
git branch -a
git switch -c feature/payment
git diff
git add .
git restore --staged <file>
git commit -m "message"
git commit --amend --no-edit
git stash
git stash list
git stash pop
git merge <branch>
git restore <file>
git revert <commit-id>
git branch -m <new-name>
git push -u origin <branch>
git tag v1.0
git push origin v1.0
```

```
docker pull ubuntu
docker image ls
docker ps
docker ps -a
docker run -it ubuntu bash
docker build -t image .
docker run -d -p 7098:8080 --name container image
docker stop container
docker start container
```

```
docker rm container
docker login
docker tag image username/image:latest
docker push username/image:latest
docker commit <container-id> username/image:latest
```

```
docker pull ubuntu
docker image ls
docker run -it ubuntu bash
```

```
# inside container
pwd
ls
echo "Hello Docker" > notes.txt
cat notes.txt
```

```
exit
```

```
docker ps -a
docker commit <container-id> lethalgod/ubuntu-custom:latest
docker image ls
docker login
docker push lethalgod/ubuntu-custom:latest
```